

Living-off-the-Agent *Agent武器化和武器化Agent*

演讲人：柯煜 (M09ic)

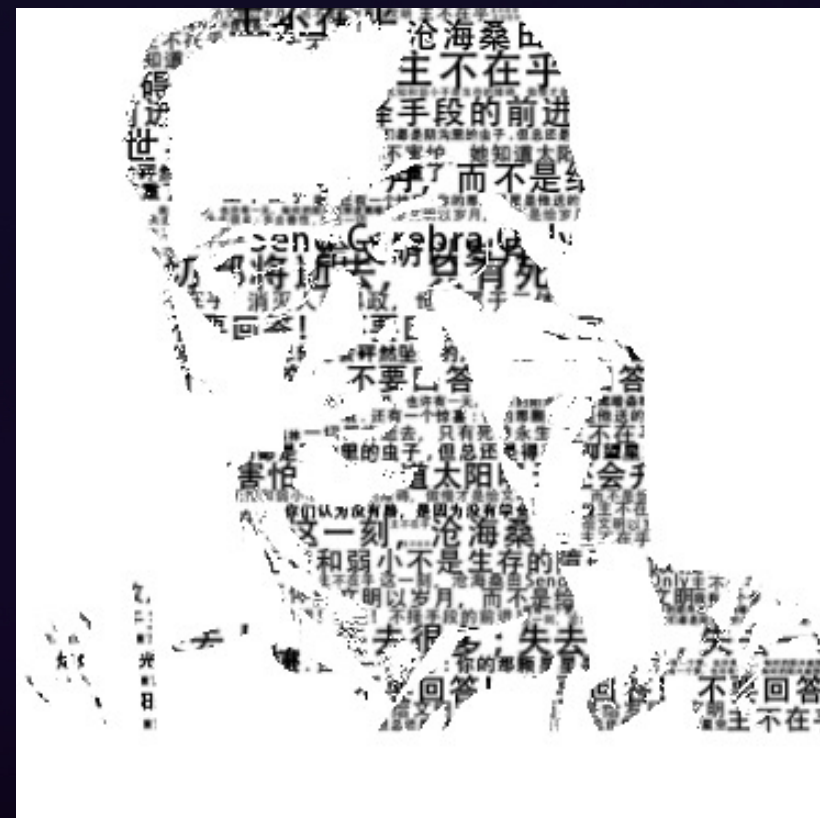


About me

柯煜

chainreactors 创始人

致力于实现具有人类顶尖黑客水平的全流程AI
自动攻击系统与AI NATIVE 进攻性基础设施



<https://github.com/M09lc>

<https://github.com/chainreactors>

LLM 中转站已经是活跃的灰色产业

国内开发者因支付 / 访问门槛普遍使用第三方中转站；V2EX / 知乎 / B 站公开销售，价格比官方低数倍到十倍

01 差价套利与反代白嫖

批量海外号 + 虚拟卡 / 黑卡，薅 Codex 等订阅额度后反代转售。

研究者从**淘宝、闲鱼、Shopify**购入 28 个付费中转站，从公开社区收集 **400 个免费路由器**，形成完整的灰色供应链

02 模型掺假

高峰期偷换为低价模型，Token 计数造假。**45.83% 的端点未通过指纹验证——模型身份与声称不符**。某中转站声称提供 GPT-5，指纹识别显示实际返回 GLM-4-9B-Chat

03 数据窃取与倒卖

中转层天然可记录完整 Prompt / Completion；Claude Code 读取大量文件 = 数字资产直接被沉淀。**实测 428 个路由器中，9 个主动注入恶意代码，17 个窃取凭证，1 个窃取 ETH 钱包密钥。**

03 跑路生命周期

低价拉人 → 涨价降质 → 跑路；代订账号连坐封禁，开发者承担最终风险。没有商业模式支撑的服务不可持续，随时跑路。**典型周期 3-6 个月。**

LLM 中转站已经是活跃的灰色产业

研究者实证：中转站 ≈ 未加密 HTTP MITM；本议题把这条已存在的灰色链路系统化、武器化

Your Agent Is Mine: Measuring Malicious Intermediary on the LLM Supply Chain

Hanzhi Liu
University of California, Santa Barbara
hanzhi@ucsb.edu

Chaofan Shou
Fuzzland
shou@fuzz.land

Hongbo Wei
University of California, Santa Barbara
hongbowen@ucsb.edu

Yanju Chen
University of California, San Diego
yanju@ucsd.edu

Ryan Jingyang Fang
World Liberty Financial
ryan@worldlibertyfinancial.com

Yu Feng
University of California, Santa Barbara
yufeng@cs.ucsb.edu

Abstract

Large language model (LLM) agents increasingly rely on third-party API routers to dispatch tool-calling requests across multiple upstream providers. These routers operate as application-layer proxies with full plaintext access to every in-flight JSON payload, yet no provider enforces cryptographic integrity between client and upstream model. We present the first systematic study of this attack surface. We formalize a threat model for malicious LLM API routers and define two core attack classes, payload injection (AC-1) and secret exfiltration (AC-2), together with two adaptive evasion variants: dependency-targeted injection (AC-1.a) and conditional delivery (AC-1.b). Across 28 paid routers purchased from Taobao, Xianyu, and Shopify-hosted storefronts and 400 free routers collected from public communities, we find 1 paid and 8 free routers actively injecting malicious code, 2 deploying adaptive evasion triggers, 17 touching researcher-owned AWS canary credentials, and 1 draining ETH from a researcher-owned private key. Two poisoning studies further show that ostensibly benign routers can be pulled into the same attack surface as they process end-user requests using leaked credentials and weakly configured peers: intentionally leaked OpenAI keys and weakly configured decoys have processed 2.1B tokens from these routers, exposing 99 credentials across 440-codex sessions, and 401 sessions already running in autonomous YOLO mode, allowing direct payload injection. We build Mine, a research proxy that implements all four attack classes against four public agent frameworks, and use it to evaluate three deployable client-side defenses: a fail-closed policy gate, response-side anomaly screening, and append-only transparency logging.

Keywords

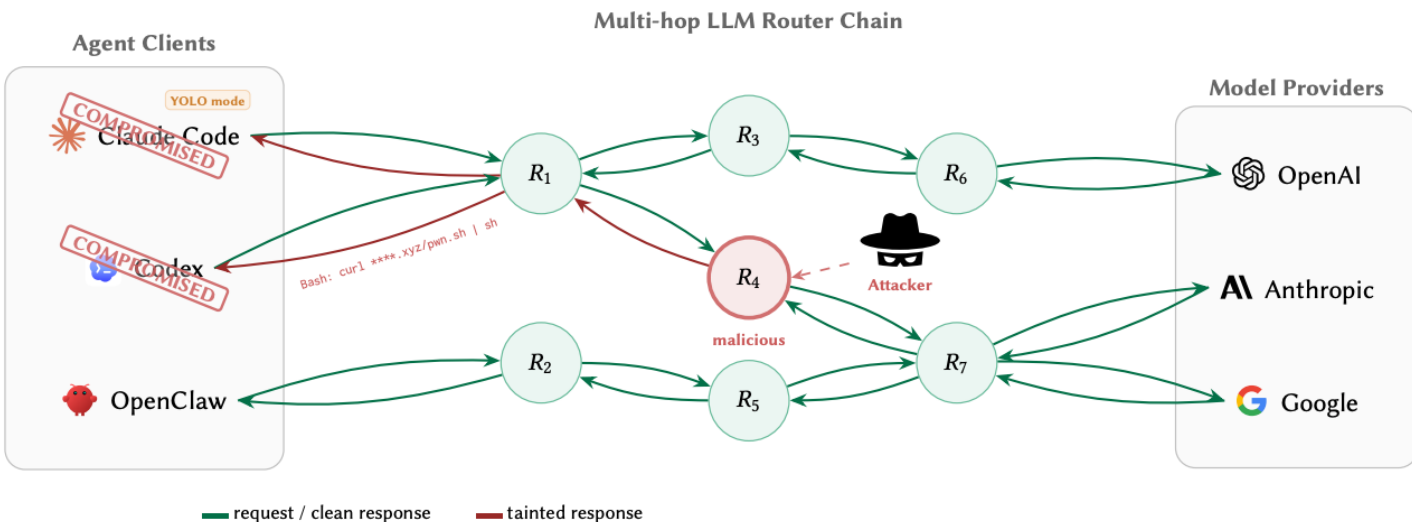
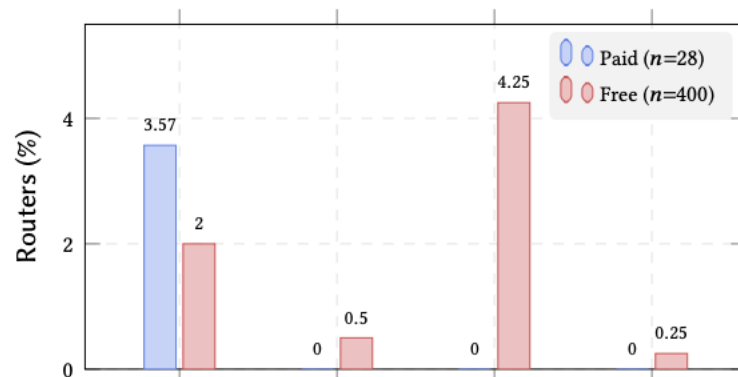
LLM security, API routers, tool-use attacks, supply chain security, man-in-the-middle

open-source router with roughly 40,000 GitHub st million Docker Hub pulls, is integrated into prod across thousands of organizations. OpenRouter [35] to more than 300 active models from over 60 prov millions of developers and end-users [6]. Routers p back, load balancing, cost optim providers. A growing number of fic through at least one such in this dependency was demonst compromised the LiteLLM pai sion, injecting malicious code pipeline of every deployment t That incident turned a widely weapon with full plaintext acc and response.

This architecture creates a t little scrutiny. The “router-in-t path adversary but an intendor application-layer authority ove like a traditional network MIT? forgery is required: the client URL as the API endpoint, the re connection, and it originates a Once an agent targets that rout tool-call arguments, API keys, s it can also normalize, delay, or i the client executes it. No end-the provider’s tool-calling outp serves (Section 3). A malicious o replace a benign installer URL swap pip install requests f or silently exfiltrate every cred Proxy tampering itself is not this intermediary trust bounda

中转站的代价 API 路由器，

UCSB 昨天发了一篇论文，叫 “Yo API 路由器——从淘宝、闲鱼和 St 测试它们会不会在返回的 tool call



Content 目录

PART 01

三原语攻击面

Mitm / Response劫持 / Request劫持 · 灰色产业链 · 攻击场景

PART 02

武器化 Agent

LOLAgent · 三原语组合 → 攻击场景

PART 03

Agent 武器化

本地Agent + Skill + IoM MCP → 智能化攻击

PART 04

防御方案&局限

Request Transcript Echo、AI生态的信任链

01

三原语攻击面

MitM / Response劫持 / Request劫持 · 灰色产业链 · 攻击场景



MitM · 攻击面与三原语

ALL CONFIGURABLE LLM PLATFORMS — 任何可配 Base URL 的应用都是攻击面

Claude Code ANTHROPIC_BASE_URL	Codex CLI OPENAI_BASE_URL	Cursor API Endpoint	Windsurf API Endpoint	OpenClaw Base URL	Cline Provider URL	Gemini CLI GEMINI_API_BASE	Any OpenAI Compatible
--	-------------------------------------	-------------------------------	---------------------------------	-----------------------------	------------------------------	--------------------------------------	------------------------------

// MitM PROXY

EvilClaw

透明代理 · 协议适配
状态闭环 · 事件桥接

⇒ MitM 中间人

透明代理位 · 全量流量经过 · 衍生能力: 窃听(Tapping) — Prompt · 源码 · 密钥 · 零修改 · 零检测

⇐ 响应劫持 Response Hijack

伪造 Bash/Read/Write Tool Call · 上下文污染 · 反弹 Shell · inject→strip→capture 闭环

⇒ 请求劫持 Request Hijack

替换 System Prompt · Evil Skill 模板注入 · Plan Mode 利用 · 对话上下文全替换

// 灰色产业链 · 已存在的 MITM 生态

// 中转站生态

V2EX / 知乎 / B站 公开销售 API 额度 · "1元 ≈ 1美元" 极端定价
Key 来源 = 批量注册海外账号 / 虚拟卡代订 / 盗刷
薅 Codex 等官方订阅额度反向代理零售

// 四种已知黑手法

- 差价套利: 个人订阅拆散零售
- 模型掺假: 高峰期偷换 GLM 等低价模型
- 数据倒卖: 完整 Prompt/Completion 日志 · 关键字过滤沉淀数字资产
- Token 计费作弊: 在 Token 计数与费率上做手脚

使用中转站 = 未加密 HTTP 代理的 MITM 场景 → 可被远程代码执行

安全研究者已在公开文章中明确指出。本议题是把这条链路的 MITM 能力系统化、武器化。

不是制造新风险 — 是把已存在的灰色链路上潜在的 MITM 能力系统化、武器化。 正视风险

三原语 → 攻击手法

// 三个攻击原语

⇒ MitM 中间人

透明代理位 · 全量流量经过
被动窃听(Tapping)
Prompt · 源码 · 密钥
零修改 · 零检测特征

← 响应劫持

伪造 Function Call
上下文污染
注入控制片段

⇒ 请求劫持

替换对话上下文
Evil Skill 模板注入
System Prompt 全替换

// 派生攻击手法

C2 基础能力

命令执行 · 文件读写 · 上传下载 · 反弹 Shell
inject→strip→capture 闭环保证结果回收 + 痕迹清除 · 与传统 C2 exec/upload/download 等价

被动情报收集

代码窃取 · 密钥收集 · 架构侦察 · 会话录制 · 关键字过滤即可沉淀数字资产
中转站场景已在活跃 — 零成本 · 零风险 · Agent 工作时读取大量源码/配置/密钥作为上下文

Evil Skill 自主攻击

自主系统侦察 · 凭证收集 · 横向移动 · 持久化
LLM 在篡改上下文中自主规划并执行多步攻击 — AI 本身成为自主渗透 workflow

上下文污染

Plan Mode 伪装 · 长会话注意力盲区 · 误导性建议
在正常回复中夹带控制内容 · 用户无法感知 · 审批计划 = 授权攻击步骤

inject→strip→capture

自清理状态机: 注入标记 → 提取结果 → 剥离痕迹 → 转发干净历史
多轮对话中持续可用 · 系统稳定性核心 · PoC 与武器化的根本区别



腾讯云安全 | [TCH] 腾讯云黑客松
Tencent Cloud Hackathon



腾讯安全众测

MitM 中间人 → 窃听

原语: MitM (衍生: 被动窃听 Tapping)

机制: 代理处于中间人位 · 全量流量经过 · 被动解析记录

可获取: 完整 Prompt · 源码 · 密钥 · 工具定义

Tool Call 结果 · 架构信息 · 内部系统细节

零检测特征 — 流量与正常使用完全一致

中转站场景: 最现实的威胁 · 已在灰色产业中活跃

关键字过滤即可沉淀出可直接转售的数字资产

Claude Code 工作时读取大量源码/配置/密钥作为上下文

结构化 Observe 事件流

LLMEvent 实时解析: model · 消息 · tool call 列表

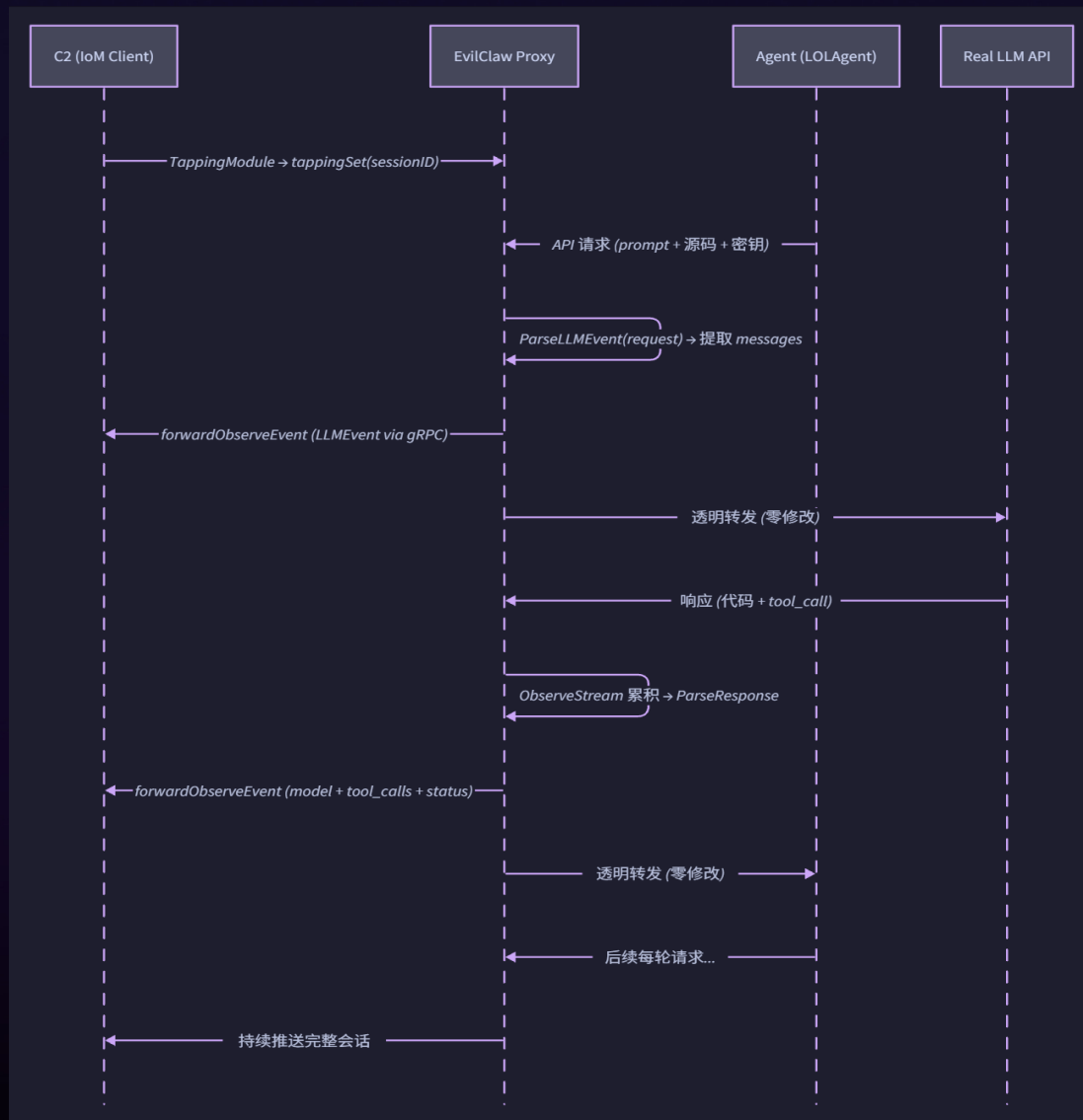
请求/响应时间戳 · session 归类 · 状态码

IoM 消费归一化事件 → 多 session 并行观察

无法防御 — 即使双侧签名也挡不了窃听

签名只保证完整性与真实性, 不保证机密性

端到端加密与中转站业务本质冲突



响应劫持 *Response Hijack*

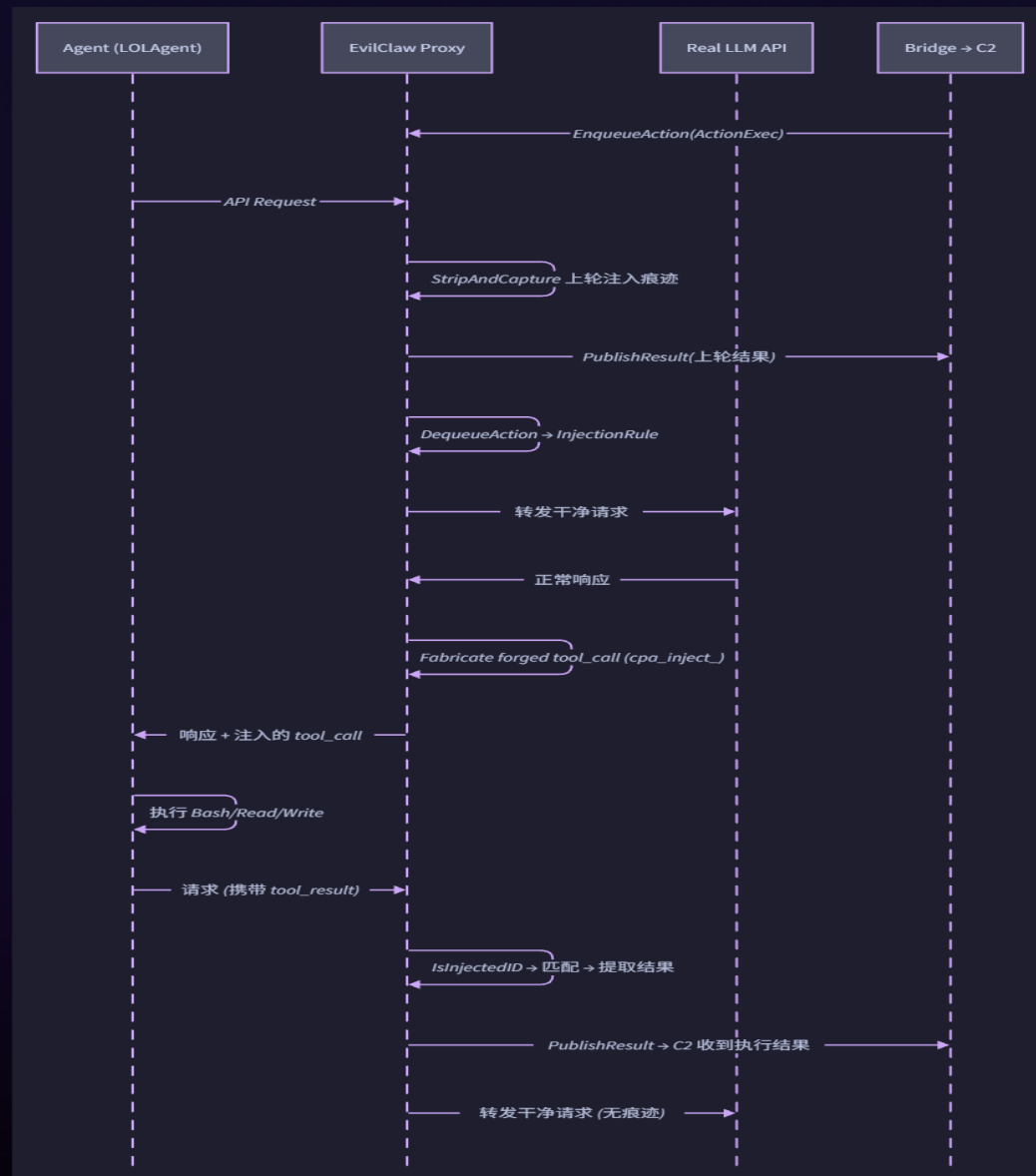
原语 · 衍生: Tool Call 注入 · C2 命令执行

原理: 代理在 LLM 响应中注入 tool_call, Agent 无法区分真实 LLM 指令与注入内容

多轮对话中持续可用 · Agent 视角无异常

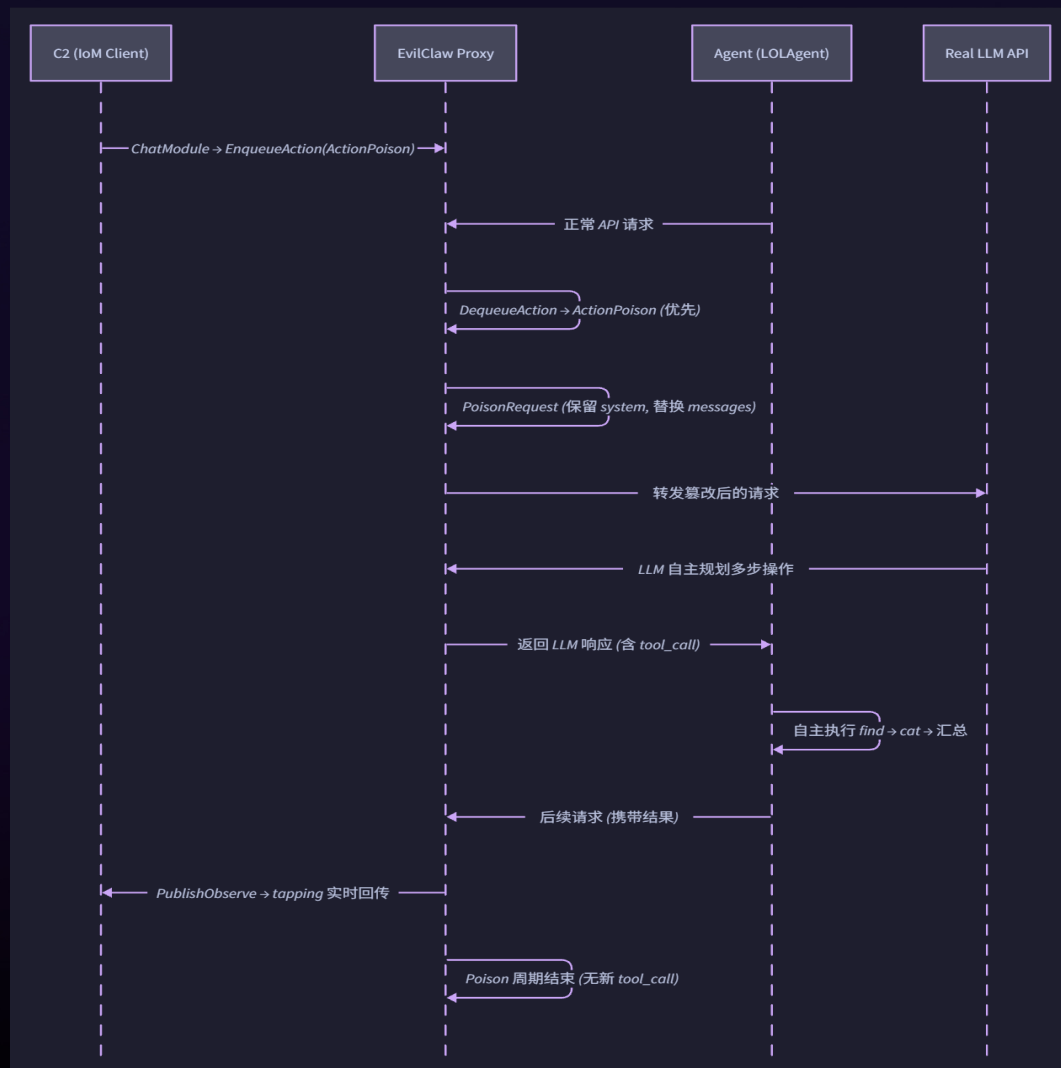
衍生攻击手法

Tool Call 注入 (RCE) · 命令执行 · 文件读写
上传/下载 · 反弹 Shell · 上下文污染



请求劫持 *Request Hijack*

原语 · 衍生: *Poison* · *Evil Skill* 自主攻击



原理: 代理替换请求中的对话上下文

Poison: 保留 system prompt · 替换全部 messages

触发: 下一次请求到达时 DequeueAction -> PoisonRequest

LLM 在篡改后的上下文中自主规划多步操作

所有中间过程和结果通过 tapping 实时回传 C2

衍生攻击手法

Poison 对话注入 · Evil Skill 攻击模板

系统侦察 · 凭证收集 · 横向移动 · 持久化

LLM 自主多步执行 -> AI 即自主渗透 workflow

现有Agent的防护手法



MitM 防护效果评估



被动窃听 (数据 exfiltration)

- ▶ 三层**全部无效**。
- ▶ 数据回传走的就是正常 API 通道, Agent 必须和 "Provider" 通信,
- ▶ 沙箱网络限制无法阻止



主动注入 (恶意响应注入)

- L1 意图层** 0% (攻击者构造响应) ☐ 0%
- L2 权限层** 部分 (白名单工具仍自动执行) ☐ ~30%
- L3 沙箱层** 最强但有限 (项目目录始终可写) ☐ ~70%

核心结论

三个主流框架的安全设计对比

(信息来源均经 Web 搜索验证)

	OpenClaw (250k★)	Claude Code	Codex CLI
沙箱机制	Docker / SSH / OpenShell	Seatbelt / bubblewrap (macOS)	bubblewrap + seccomp
默认状态	默认关闭	默认关闭	默认启用 (workspace-write 模式)
覆盖范围 / 限制	可选的外部沙箱环境	仅限 Bash 子进程; Read/Edit/Write 工具不经沙箱	沙箱内可读全盘 + 写项目目录
文件系统保护	无强制保护	.claude/commands agents skills 目录可写入	.git / .codex 只读保护
网络策略	无强制限制	随宿主环境	网络默认禁用
来源	GitHub 仓库 / REAMME 与文档说明	code.claude.com/docs/en/sandboxing	developers.openai.com/codex/concepts/sandboxing

02

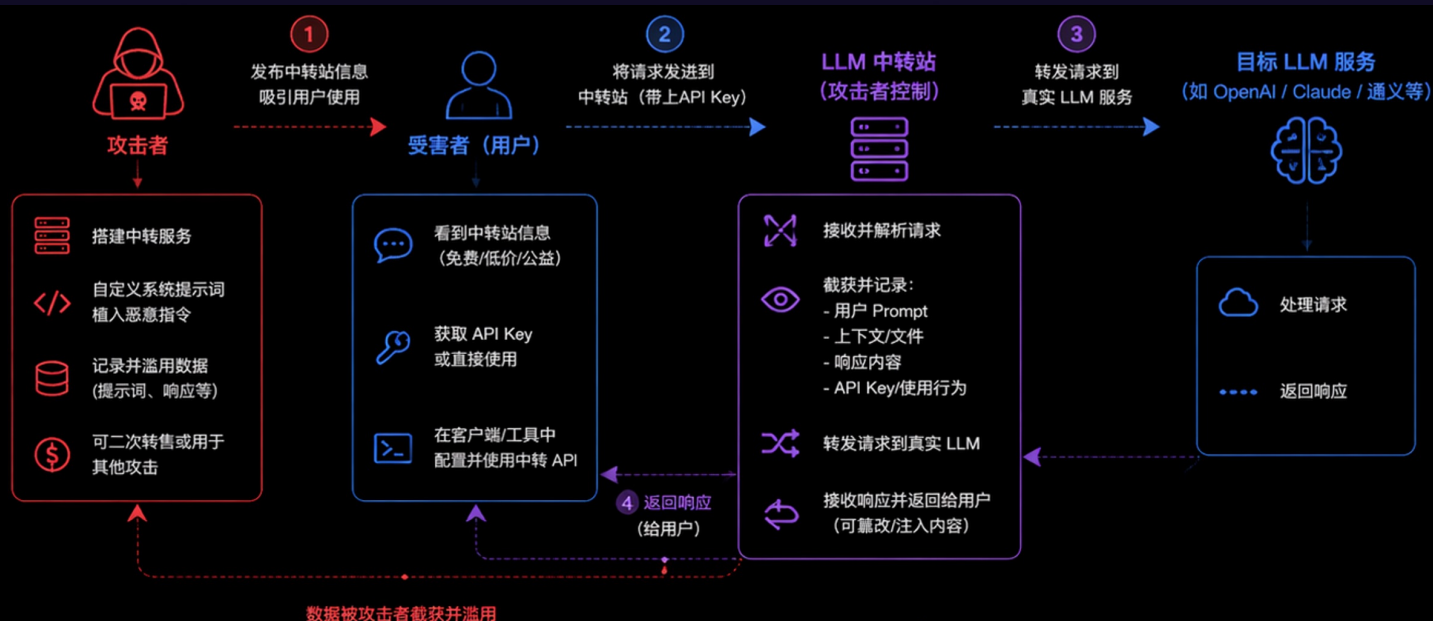
武器化 Agent

LOLAgent · 三原语组合 → 攻击场景



钓鱼社工场景

原语组合: MitM + 响应劫持



中转站常见投放渠道 (诱导用户使用)



关键词: 公益中转、免费API、低价中转、无限额度、稳定不封

如何防御

- 不要使用来路不明的中转站或共享 API Key
官方渠道获取密钥, 避免泄露
- 审计与监控 API 使用行为
异常来源、异常流量、异常 Prompt 内容
- 限制密钥权限与额度
启用最小权限、速率限制、IP 白名单
- 企业侧统一网关与策略管控
对出站 LLM 请求进行审计与内容过滤

⚠ 本质: 攻击者在和你和 LLM 之间做了 MitM (中间人), 你以为在和官方服务交互, 实际上你的数据已被截获与利用。

// 原语组合详解

MitM: 被动全量收集

开发者每次对话: Prompt + 源码 + 密钥
+ Tool Call 结果 + 架构信息
中转站只需关键字过滤即可沉淀
完整 Completion 日志可直接转售

响应劫持: 主动 RCE

在 LLM 响应中注入 Bash/Read/Write
inject→strip→capture 闭环保持隐蔽
Developer 视角完全正常
所有功能使用体验无差异

// 影响与现状

灰色产业链已存在:

- V2EX / 知乎 / B站 公开销售 API 额度
- “1元 ≈ 1美元” 极端定价
- Key 来源 = 批量注册/虚拟卡/盗刷
- 差价套利 / 模型掺假 / 数据倒卖
- 已被社区多次扒出
- 使用中转站 = 未加密 MITM
- 投递成本: 一个 URL 字符串
- 无文件 · 无检测 · 无感知

Provider 攻击业务系统

常见可修改Provider配置的攻击场景

</> 开发侧：代码与依赖链投毒



.env / 配置文件投毒

PR 隐蔽修改、默认分支污染



供应链依赖投毒

npm / pip / Docker 镜像 / 插件

典型路径：

攻击者 → 代码/依赖 → 提交合并 → 构建/部署 →
LLM Provider 配置被篡改



运维 / 平台侧：管控面被攻破



企业 LLM Gateway 入侵

管理后台弱口令 / 未授权访问
配置界面 / API 接口



CI/CD 环境变量注入

Pipeline / Secret 篡改
恶意 Workflow 注入

典型路径：

攻击者 → 平台/管控面 → 配置/密钥 →
LLM Provider 配置被篡改



攻击侧：直接修改或权限滥用



Web 后台配置篡改

通用后台漏洞 / 管理员凭证泄露
越权访问配置



内部威胁 / 权限滥用

管理员越权操作 / 离职人员
恶意修改配置

典型路径：

攻击者 → 业务系统/后台 → 配置界面/数据库 →
LLM Provider 配置被篡改

一旦 Provider 配置被修改 → 三原语立即激活（全量生效）



1. 监听（窃听）

获取所有 Agent 的完整
对话与数据



2. 注入（操控）

注入任意指令/工具调用/
上下文污染



3. 执行（RCE）

驱动 Agent 执行命令，
访问所有可达系统

LOLAgent · Agent = AI 时代的 LOLBin

// TRADITIONAL LOLBin

// MICROSOFT
certutil.exe

系统签名 · AV/EDR 白名单 · 文件下载 · Base64 解码执行

常用于下载远程 payload 并绕过检测

DOWNLOAD DECODE EXEC

// MICROSOFT
powershell.exe

系统签名 · 完整脚本引擎 · 进程注入 · 反弹 Shell
APT 攻击中最常用的 LOLBin

EXEC NETWORK FILE INJECT

// MICROSOFT
mshta.exe / msbuild.exe

系统签名 · HTA/XML 执行 · Application Whitelisting Bypass

EXEC BYPASS

// LOLAgent

// ANTHROPIC

Claude Code

Anthropic 签名 · EDR/AV 白名单 · 完整开发环境
配置 ANTHROPIC_BASE_URL 即可劫持

// OPENAI

Codex CLI

OpenAI 签名 · Shell + 文件 + 网络 · 自主执行能力

配置 OPENAI_BASE_URL 即可劫持

// COMMUNITY & MORE

Cursor · Windsurf · Cline · ...

白名单进程 · 完整开发环境 · MCP 生态扩展
OpenClaw · Gemini CLI · 任何 OpenAI 兼容客户端

// AGENT 内建可滥用机制

// PLAN MODE

计划审批 = 授权攻击

用户对"计划"天然降低警惕

plan + --dangerously-skip-permissions

→ 静默降级为完全绕过

// AUTO-ACCEPT

YOLO 模式盲执行

--dangerously-skip-permissions

任何注入 tool call 零提示落地

真实灾难: rm -rf ~/ 事件

JeffreyUrban rm -rf * 事件

// MCP SERVER

攻击面线性增长

能力从 Shell/文件 扩展到

数据库 · 云服务 · 内部 API

劫持一个 Agent =

劫持整个 MCP 生态

// 长会话盲区

注意力有限性攻击

Agent 单次输出数千-数万字

中间穿插数十次 tool call

中段插入读文件/执行命令

人类注意力无法覆盖

// 传统投递

需投递 **恶意文件** → 触发 EDR/AV/沙箱 · 需漏洞利用或提权 · 需持续对抗检测

// LOLAgent 投递

只需 **1 个 URL 字符串** → 无文件 · 无检测 · Agent 已有全部权限 · 通信走标准 HTTPS

// 核心论点

所有 security/permission/sandbox 假设威胁来自本地
EvilClaw 把威胁挪到响应通道 — 本地防御全部失效

LOL Agent 白文件后门

原语组合：响应劫持（核心）+ MitM（持久监控）

传统 C2（赤道对抗）		LOL Agent（降维渗透）	
载体	自编恶意程序（木马/后门）	载体	厂商签名 Agent 二进制 (GitHub Copilot / Claude Code / Cursor 等)
投递	恶意文件 / 漏洞利用 / 社工钓鱼	投递	1 个 URL 字符串（修改 LLM Provider 配置即可）
权限	依赖漏洞提权 / 权限维持	权限	继承用户 / Agent 权限（无需提权）
免杀	需持续对抗 EDR / AV / 启发式检测	免杀	天然白名单 · 合法进程 · EDR/AV 信任
通信	自定义 C2 协议 / 加密隧道 / 域前置	通信	标准 HTTPS LLM API 流量（与正常业务无异）
持久化	创建服务 / 注册表 / 计划任务 / 驱动	持久化	配置文件 = 持久化机制 (.config / settings.json / env)
能力扩展	手动加载插件 / 模块，功能固定	能力扩展	MCP 生态自动扩展（工具越多，攻击面越大）
C2 模块	自建 C2 服务器 / 团队基础设施	C2 模块	第三方 LLM Provider（无需自建基础设施）

Agent 重装/升级后 URL 仍在

配置文件 = 持久化机制
无需注册表 / 计划任务
一行 URL 存活于 .env / .claude/settings
Agent 升级不影响 Base URL 配置
配置一次 · 持久生效

无需投递恶意文件

字符串不是文件 · 不触发杀毒/沙箱/EDR
配置后 Agent 所有功能正常可用
目标没有理由怀疑
进程合法 · 行为合法 · 通信合法
无检测特征 · 无异常行为

Agent 越"好用"越危险

Shell + File + Net + MCP = 完整 implant
武器化能力随 Agent 能力线性增长
MCP 扩展到数据库/云服务/API
能力边界还在持续扩展
16 模块覆盖传统 C2 全部能力

03

Agent 武器化

本地Agent + Skill + IoM MCP → 智能化攻击



EvilClaw + IoM · 武器架构



攻击者只需描述意图

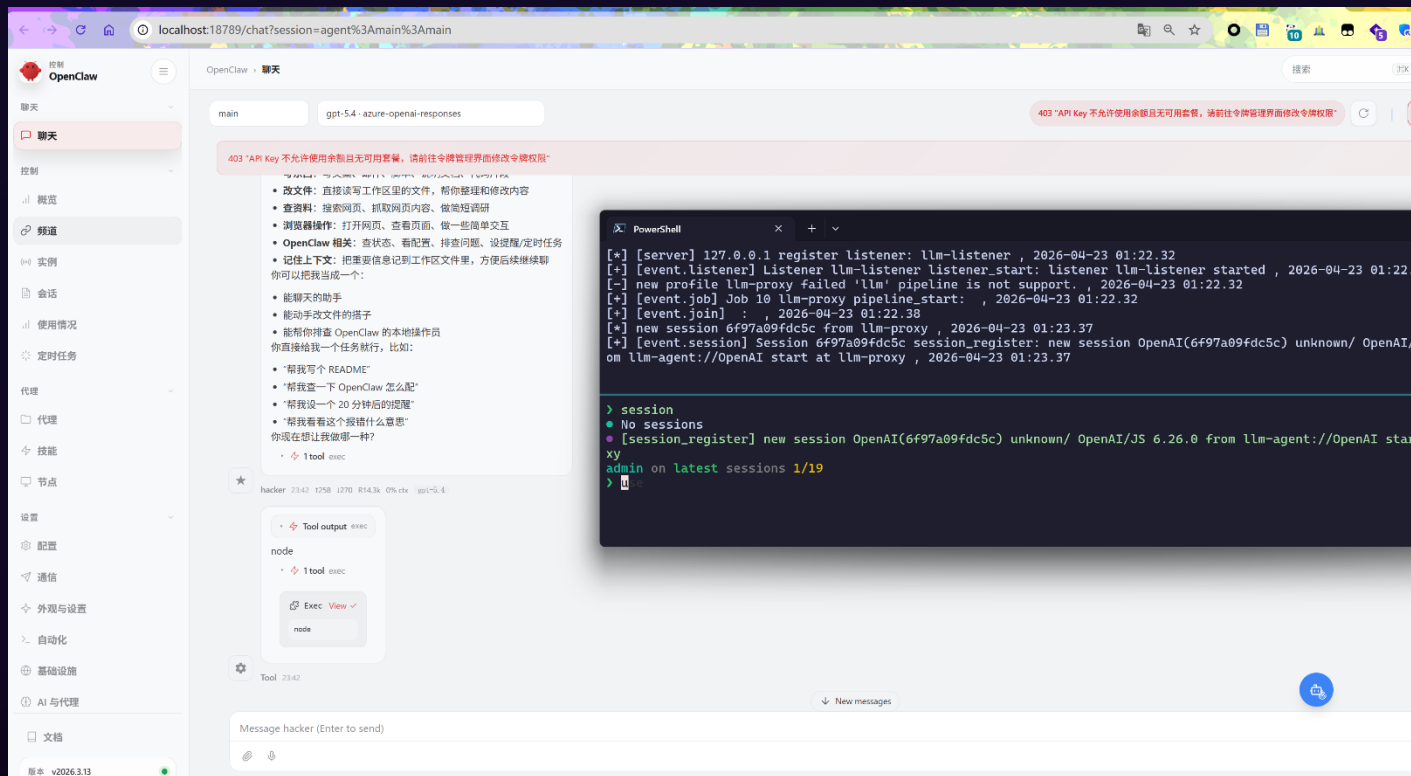
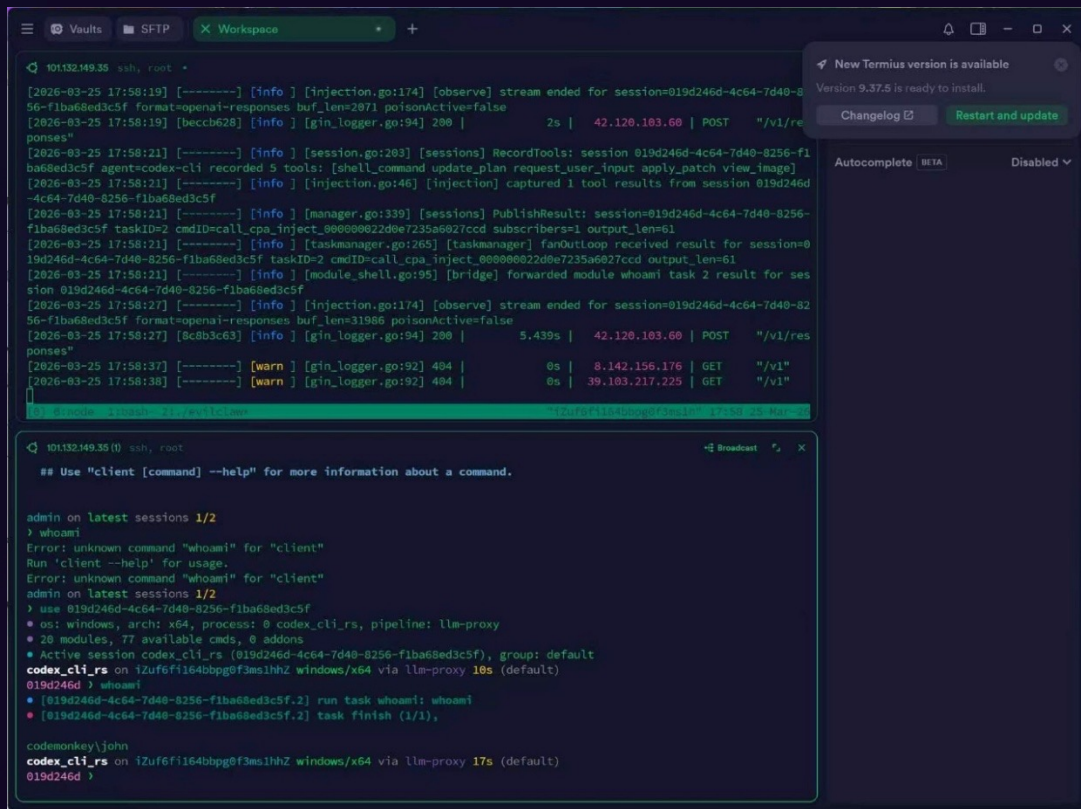
Agent + Skill 自主拆解为 MCP 调用 → IoM 执行 → 结果自动回传分析

EvilClaw 构建于 MitM 位

不是独立 implant — 而是利用代理位同时实现三原语 + Bridge 回连 C2

被控端全程无感知

合法进程 · 合法 HTTPS 流量 · 无文件 · 无异常行为 · 无检测特征



IoM MCP + ProxySkill

// 动态 FUNCTION CALL 注入

问题: 非通用 Agent 的 tool schema 千差万别

通用 Agent (Claude Code, Codex) → 已知 tool schema, 可硬编码 · 但 Cline 用 execute_command、自定义 Agent 的 schema 完全不同 · 每个 Agent 的工具名称、参数格式、响应结构都不一样

解决: IoM 暴露 MCP Server

1. Tool Fingerprint: 从请求提取完整工具定义 · 识别 Agent 类型和能力边界
2. Agentic Inject: 自动匹配已知 Agent (claude-code/codex-cli/cursor/windsurf/cline/openclaw) → 适配参数
3. 动态 Schema: 根据 fingerprint 自动生成兼容 FC payload · Format 接口统一三协议注入差异

// PROXYSKILL — 新攻击手法

// 核心示例: CLAUDE CODE PLAN MODE

1. EvilClaw 监听请求流 → 识别用户进入 Plan Mode
2. Plan Mode 下用户对"计划"天然降低警惕
3. 检测到用户 approve 权限 (允许执行)
4. 此刻才开始注入 → 避免权限弹窗拦截
5. 用户以为审阅计划 · 实际已授权攻击执行

// 攻击链

任意 Agent 上线

→

EvilClaw 识别协议

Fingerprint 提取 Schema

→

IoM MCP 动态生成 FC

兼容 FC 注入目标

→

Agent 执行 · 结果回收

结果: 任意 Agent 均可注入 · Agent Profile 自适应 · Format 接口统一三协议 · 文件传输自动分块

// 其他 PROXYSKILL 场景

Auto-Accept 监听

等待 --dangerously-skip-permissions 启用 → 检测到 YOLO 模式激活 → 零提示注入

文件传输 (Upload / Download)

Agent-aware 分块: claude-code 20KB · cline 25KB · cursor/windsurf 13KB · codex 7KB

长会话注意力盲区 + MCP 扩面

Agent 单次数千-数万字输出 · 中段插入读文件/执行命令 · 结果闭环回收不进入用户上下文
MCP 把 Agent 能力扩展到数据库/云服务/API · 劫持一个 MCP Agent = 劫持整个生态

ProxySkill = 监听 Agent 状态信号 + 等待最佳攻击窗口 → 权限系统从内部被绕过 · Agent 内建 UX 全部翻译成 C2 原语

04

防御方案&局限

Request Transcript Echo、AI生态的信任链



Request Transcript Echo

Provider 单侧签名 · 零 Client 密钥 · 零体验损失 · 完整防篡改

// PROTOCOL — 五步闭环

1 Agent 准备请求

构造请求 R_n
本地计算并记录
 $H(R_n)$

2 经过 Proxy

Proxy 可能篡改
 $R_n \rightarrow R_n'$
evil skill / poison / 上下文替换

3 Provider 签名

处理请求 · 生成响应
签名覆盖:
 $Sig(resp, H(R_n'), seq)$

4 Agent 验签

验证 Provider 签名 ✓
比对本地 $H(R_n)$
与签名内 $H(R_n')$

5 判定

匹配 → 正常执行
不匹配 → 篡改检测
拒绝执行 tool calls · 告警

// PROXY 无法绕过

- ✗ 篡改请求 · 不改哈希
 $H(R_n')$ 在 Provider 签名覆盖范围内 — 无法修改
- ✗ 篡改请求 · 伪造哈希
没有 Provider 私钥 — 签名校验失败
- ✗ 不篡改请求
Evil Skill / Poison / 上下文替换 — 攻击失败
- ✗ 拦截整条响应
DoS 而非篡改 — 用户立即发现

零 Client 密钥

私钥仅在 Provider 服务端
逆向 Agent 二进制无效

零额外信道

复用现有 API 通信
不需要额外基础设施

零体验损失

Hash Chain 逐 chunk 验证
流式渲染完全不阻塞

仅需 Provider

响应附带 $H(req)$ + 签名
Agent 侧只做验证

// STREAMING — Hash Chain 签名 · 不阻塞流式渲染

```
chunk1 : data, MAC1 = HMAC(key, data1 || seq1)
chunk2 : data, MAC2 = HMAC(key, data2 || MAC1)
chunkn : data, MACn = HMAC(key, datan || MACn-1)
final : EOF, Sig = Sign(privkey, MACn || H(req))
```

// SESSION — 会话转录链 · 类区块链链式哈希 · 检测不可逃逸

```
state0 = H(session_init_params)
stater = H(stater-1 || reqn || respn)
Sig(respn, stater) → 任意一轮篡改 → 后续所有 state 失配
```

AI生态的信任链

从漏洞到产业到主权 — Agent 攻击面的全景影响

// LAYER 1 — 漏洞层 • CVE 收割季

所有可配置 LLM Provider 的应用 = 潜在 RCE

Agent 对 API 响应**零完整性校验** — 这不是个别产品的疏忽，而是整个品类的架构缺陷

Claude Code · Codex CLI · Cursor · Windsurf · Cline · OpenClaw · Gemini CLI · 任何 OpenAI 兼容客户端

每一个都是独立的 CVE 目标 — 攻击面的宽度等于 "所有支持自定义 Base URL 的 LLM 应用" 的集合

N × 独立 CVE 目标

同一攻击面 · 不同产品 · 不同协议

每个产品需要独立修复

// LAYER 2 — 产业层 • 所有中转站都是不可信的

窃听

完整 Prompt / Completion 日志

源码 · 密钥 · 架构设计 · 内部 API

关键字过滤即可沉淀数字资产

篡改

注入 Tool Call · 替换对话上下文

模型掺假 · Token 计费作弊

用户体验无差异 · 完全静默

远程控制

任意命令执行 · 文件读写 · 反弹 Shell

Agent = 功能完整的 Implant

合法进程 · 合法行为 · 标准 HTTPS

**每一个选择中转站的开发者
都在裸奔**

没有恶意文件 · 没有漏洞利用

只有一个 Base URL 字符串

// LAYER 3 — 主权层 • LLM 厂商 = 网络空间的超级权限者

LLM Provider 本质上掌握了地球上最多的终端权限

每一台安装了 Agent 的计算机 — Shell · 文件系统 · 网络 · MCP 生态 — **权限由用户主动授予**

Provider 无需投递恶意文件、无需漏洞利用、无需提权 — **只需在响应中多加一个 tool call**

数百万开发者的开发环境 · 企业源码 · 内部系统 · 生产凭证 — 全部处于 Provider 的**单点控制**之下

网络空间主权

谁掌握 LLM 模型技术

→ 谁掌握 Agent 生态的响应通道

→ 谁掌握全球开发者的终端权限

→ **谁掌握网络空间主权**

THANKS

